

Mujoco MPC



MIT Robotics Seminar

Yuval Tassa
DeepMind

***Predictive Sampling:
Real-time behavior
synthesis with MuJoCo***

December 2, 2022

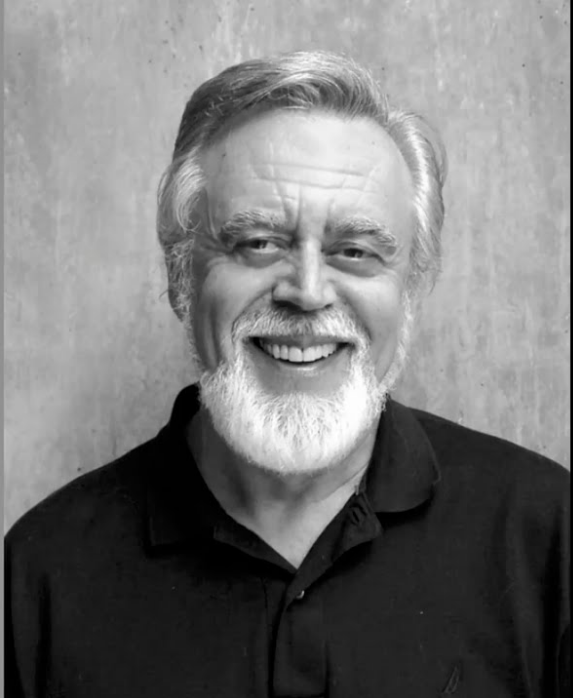
Seminar Sponsored by:



**Model Predictive Control and
Reinforcement Learning:
A Unified Framework Based on
Dynamic Programming**

Dimitri Bertsekas

Special Guest Lecture
The 8th IFAC Conference on Nonlinear Model Predictive Control
(NMPC 2024)
August 21-24, 2024
Kyoto, Japan

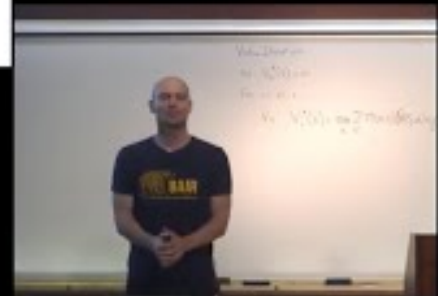


<https://people.eecs.berkeley.edu/~pabbeel/cs287-fa12/slides/LQR.pdf>

CS287 Advanced Robotics
Lecture 4 (Fall 2019)

Function Approximation

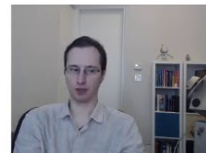
Pieter Abbeel
UC Berkeley EECS



Linear case: LQR

$$\min_{\mathbf{u}_1, \dots, \mathbf{u}_T} c(\mathbf{x}_1, \mathbf{u}_1) + c(f(\mathbf{x}_1, \mathbf{u}_1), \mathbf{u}_2) + \dots + c(f(f(\dots)), \mathbf{u}_T)$$

$$\underbrace{f(\mathbf{x}_t, \mathbf{u}_t) = \mathbf{F}_t \begin{bmatrix} \mathbf{x}_t \\ \mathbf{u}_t \end{bmatrix} + \mathbf{f}_t}_{\text{linear}} \quad \underbrace{c(\mathbf{x}_t, \mathbf{u}_t) = \frac{1}{2} \begin{bmatrix} \mathbf{x}_t \\ \mathbf{u}_t \end{bmatrix}^T \mathbf{C}_t \begin{bmatrix} \mathbf{x}_t \\ \mathbf{u}_t \end{bmatrix} + \begin{bmatrix} \mathbf{x}_t \\ \mathbf{u}_t \end{bmatrix}^T \mathbf{c}_t}_{\text{quadratic}}$$



Nonlinear case: DDP/iterative LQR

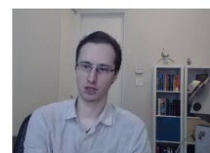
Linear-quadratic assumptions:

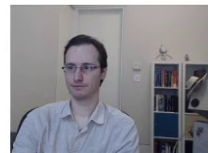
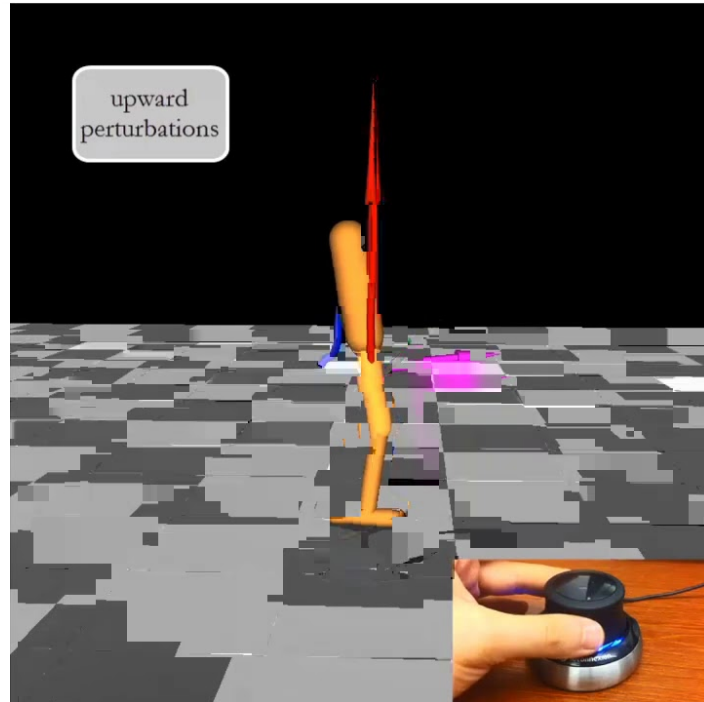
$$f(\mathbf{x}_t, \mathbf{u}_t) = \mathbf{F}_t \begin{bmatrix} \mathbf{x}_t \\ \mathbf{u}_t \end{bmatrix} + \mathbf{f}_t \quad c(\mathbf{x}_t, \mathbf{u}_t) = \frac{1}{2} \begin{bmatrix} \mathbf{x}_t \\ \mathbf{u}_t \end{bmatrix}^T \mathbf{C}_t \begin{bmatrix} \mathbf{x}_t \\ \mathbf{u}_t \end{bmatrix} + \begin{bmatrix} \mathbf{x}_t \\ \mathbf{u}_t \end{bmatrix}^T \mathbf{c}_t$$

Can we *approximate* a nonlinear system as a linear-quadratic system?

$$f(\mathbf{x}_t, \mathbf{u}_t) \approx f(\hat{\mathbf{x}}_t, \hat{\mathbf{u}}_t) + \nabla_{\mathbf{x}_t, \mathbf{u}_t} f(\hat{\mathbf{x}}_t, \hat{\mathbf{u}}_t) \begin{bmatrix} \mathbf{x}_t - \hat{\mathbf{x}}_t \\ \mathbf{u}_t - \hat{\mathbf{u}}_t \end{bmatrix}$$

$$c(\mathbf{x}_t, \mathbf{u}_t) \approx c(\hat{\mathbf{x}}_t, \hat{\mathbf{u}}_t) + \nabla_{\mathbf{x}_t, \mathbf{u}_t} c(\hat{\mathbf{x}}_t, \hat{\mathbf{u}}_t) \begin{bmatrix} \mathbf{x}_t - \hat{\mathbf{x}}_t \\ \mathbf{u}_t - \hat{\mathbf{u}}_t \end{bmatrix} + \frac{1}{2} \begin{bmatrix} \mathbf{x}_t - \hat{\mathbf{x}}_t \\ \mathbf{u}_t - \hat{\mathbf{u}}_t \end{bmatrix}^T \nabla_{\mathbf{x}_t, \mathbf{u}_t}^2 c(\hat{\mathbf{x}}_t, \hat{\mathbf{u}}_t) \begin{bmatrix} \mathbf{x}_t - \hat{\mathbf{x}}_t \\ \mathbf{u}_t - \hat{\mathbf{u}}_t \end{bmatrix}$$





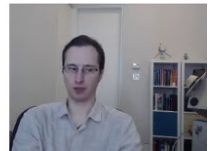
From the UCB cs285 deep RL class:

Linear case: LQR

$$\min_{\mathbf{u}_1, \dots, \mathbf{u}_T} c(\mathbf{x}_1, \mathbf{u}_1) + c(f(\mathbf{x}_1, \mathbf{u}_1), \mathbf{u}_2) + \dots + c(f(f(\dots)), \mathbf{u}_T)$$

$$\underbrace{f(\mathbf{x}_t, \mathbf{u}_t) = \mathbf{F}_t \begin{bmatrix} \mathbf{x}_t \\ \mathbf{u}_t \end{bmatrix} + \mathbf{f}_t}_{\text{linear}}$$

$$\underbrace{c(\mathbf{x}_t, \mathbf{u}_t) = \frac{1}{2} \begin{bmatrix} \mathbf{x}_t \\ \mathbf{u}_t \end{bmatrix}^T \mathbf{C}_t \begin{bmatrix} \mathbf{x}_t \\ \mathbf{u}_t \end{bmatrix} + \begin{bmatrix} \mathbf{x}_t \\ \mathbf{u}_t \end{bmatrix}^T \mathbf{c}_t}_{\text{quadratic}}$$



MPC is model predictive controller. It formulates systems as a general linear system with quadratic constraints and uses RL to provide a solution.

$$\min_{u_1, u_2, \dots, u_T} \sum_{t=1}^T c(x_t, u_t) \text{ s.t. } x_t = f(x_{t-1}, u_{t-1})$$

We want to minimize the above expression, take derivatives and set to 0.

x_t = state

u_t = action

This is a different notation:

s_t = state

a_t = action

For a RL problem we maximize rewards vs minimizing cost. Basic principles the same.

We have to incorporate dynamics and cost constraints into the above.

We can change the constraint equation into an unconstrained problem using substitution:

$$\min_{u_1, u_2, \dots, u_T} c(x_1, u_1) + c(f(x_1, u_1), u_2) + \dots + c(f(f(\dots)), u_T)$$

We need partial derivatives of f and c wrt x_t and u_t .

$$\text{need } \frac{df}{dx_t}, \frac{df}{du_t}, \frac{dc}{dx_t}, \frac{dc}{du_t}$$

Compute these using NN back prop. First order gradient descent works poorly. Use a second order method like newton's method. First order gradient methods will leave you with vanishing or exploding gradients.

The second order method is a Riccati difference equation.

Given a linear system:

$$x_{t+1} = Ax_t + Bu_t$$

or in continuous time:

$$\dot{x} = Ax + Bu$$

with quadratic cost:

$$J = \sum_t (x^T Q x_t + u^T R u^T)$$

LQR gives a feedback law:

$$u_t = -Kx_t \text{ where } K \text{ comes from the Riccati difference equations}$$

Shooting methods vs collocation:

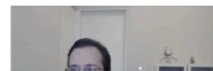
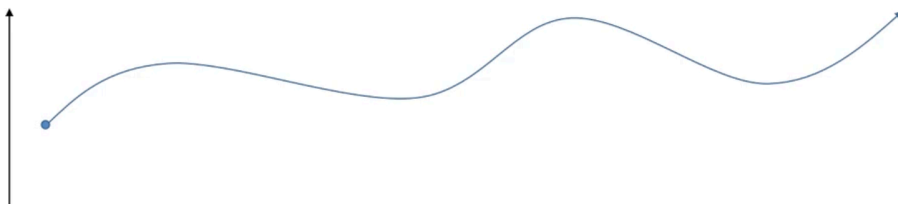
Shooting method refers to the boundary conditions on the DE with an initial condition at $t=0$ and $t=T$, the end. These 2 boundary conditions are at the beginning and end of the trajectory.

There is a dot at the beginning of the trajectory which corresponds to a boundary condition. The YT video defn may be different; either missing the $t=T$ boundary condition or left off by mistake

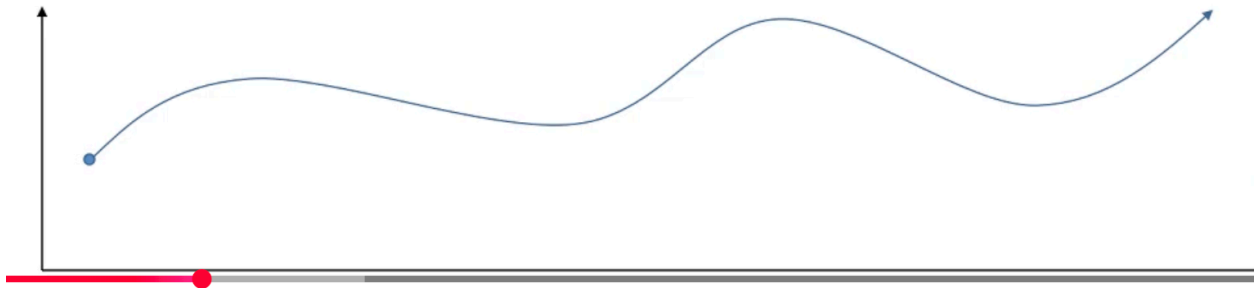
Shooting methods vs collocation

shooting method: optimize over actions only

$$\min_{\mathbf{u}_1, \dots, \mathbf{u}_T} c(\mathbf{x}_1, \mathbf{u}_1) + c(f(\mathbf{x}_1, \mathbf{u}_1), \mathbf{u}_2) + \dots + c(f(f(\dots)), \mathbf{u}_T)$$



There are many trajectories which fit the constraint. There is a high sensitivity to different solutions; different trajectories



With animation. Hard to see different end conditions. 2 graphs not the same.

Says first order methods like SGD have small and large eigenvalues which lead to numerical stability issues, either a vanishing or exploding gradient down the road.

Shooting Methods

Predictive Sampling / No derivative

Differential equation connection

Consider the linear ODE

$$\dot{x} = Ax.$$

Try a solution of the form

$$x(t) = e^{\mu t} v.$$

Then

$$\dot{x}(t) = \mu e^{\mu t} v$$

and

$$Ax(t) = A(e^{\mu t}v) = e^{\mu t}Av.$$

For the ODE to hold, we need

$$\mu e^{\mu t}v = e^{\mu t}Av.$$

Cancel $e^{\mu t}$:

$$Av = \mu v.$$

So μ must be an eigenvalue and v its eigenvector.

That means each eigenpair gives a solution mode:

$$x_i(t) = e^{\lambda_i t}v_i.$$

So the full solution is a combination of modes:

$$x(t) = \sum_i c_i e^{\lambda_i t}v_i.$$

Why this matters for stability

Each mode behaves according to its eigenvalue.

Continuous time

For

$$\dot{x} = Ax,$$

mode v_i evolves as

$$e^{\lambda_i t}v_i.$$

- if $\operatorname{Re}(\lambda_i) < 0$, that mode decays
- if $\operatorname{Re}(\lambda_i) > 0$, that mode grows
- if $\operatorname{Re}(\lambda_i) = 0$, it neither decays nor grows purely exponentially

So stability depends on the real parts of the eigenvalues.

Discrete time

For

$x_{k+1} = Ax_k$, the mode evolves as

$$x_k = \lambda_i^k v_i.$$

- if $|\lambda_i| < 1$, that mode decays
- if $|\lambda_i| > 1$, it grows
- if $|\lambda_i| = 1$, it persists/oscillates

Why “mode” is a natural word

In vibrations, PDEs, and control, a “mode” means a basic pattern of behavior.

Examples:

- one bending pattern of a beam
- one oscillation shape of a string
- one decoupled direction of a state-space system

Eigenvectors are those basic patterns. Eigenvalues tell how fast each pattern grows, decays, or oscillates.

Eigenvectors are called modes because they are special directions that evolve independently under a linear system. In differential equations like $\dot{x} = Ax$, each eigenvector v_i produces a solution mode $e^{\lambda_i t} v_i$, and the eigenvalue tells whether that mode grows, decays, or oscillates.

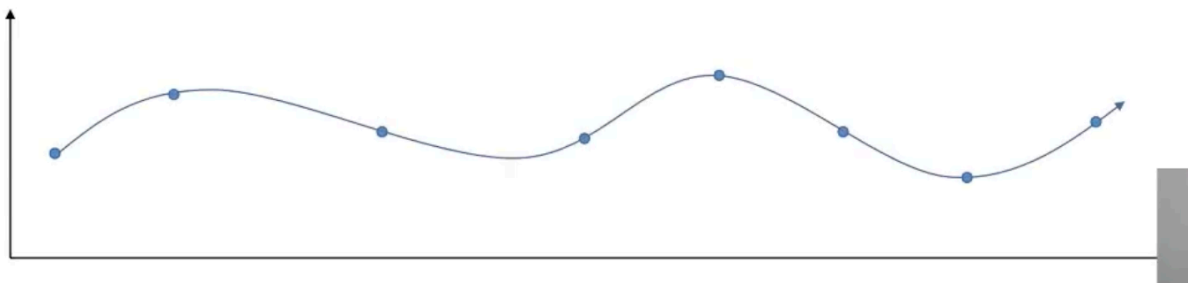
Collocation:

Optimize over states and actions with constraints vs. shooting methods which optimize over actions only.

$$\min_{u_1, u_2, \dots, u_T, x_1, x_2, \dots, x_T} = \sum c(x_t, u_t) \text{ s.t. } x_t = f(x_{t-1}, u_{t-1})$$

This has more constraints, using both the actions and states.

This is modeled as dots on the trajectory. Instead of 1 dot there are many, bounding the trajectory.



There are different methods for enforcing constraints, some act as relaxations, some are exact, some act as linearizations, they all affect the difficulty of the problem.

Collocation methods are more numerically well conditioned. Not discussed. Need additional info here(AA classes). Good project ideas.

Common collocation methods in optimal control include:

Trapezoidal collocation

Enforces dynamics with the trapezoid rule:

$$x_{k+1} - x_k - \frac{h}{2} (f(x_k, u_k) + f(x_{k+1}, u_{k+1})) = 0$$

Hermite–Simpson collocation

Uses endpoints plus a midpoint, often more accurate than trapezoidal:

$$x_{k+1} - x_k - \frac{h}{6} (f_k + 4f_{k+\frac{1}{2}} + f_{k+1}) = 0$$

This is one of the most common direct collocation methods.

Midpoint collocation

Uses the midpoint dynamics:

$$x_{k+1} - x_k - h f(x_{k+\frac{1}{2}}, u_{k+\frac{1}{2}}) = 0$$

Runge–Kutta collocation

Based on implicit RK schemes. Examples include:

- implicit midpoint
- Gauss–Legendre collocation
- Radau collocation
- Lobatto collocation

These are widely used in more accurate or higher-order transcriptions.

Pseudospectral collocation

Uses global polynomials and collocation at special nodes. Common examples:

- Legendre–Gauss
- Legendre–Gauss–Radau
- Legendre–Gauss–Lobatto
- Chebyshev pseudospectral methods

These are popular for high-accuracy trajectory optimization.

Here is a small **Hermite–Simpson direct collocation** example in Python for a double integrator:

```
\dot x_1 = x_2, \quad \dot x_2 = u
```

CartPole:

```

import numpy as np
from scipy.optimize import minimize

def dynamics(x, u):
    """
    Double integrator dynamics.
    x = [position, velocity]
    u = scalar acceleration
    """
    return np.array([x[1], u])

def unpack_decision_vars(z, N, n, m):
    """
    z contains:
    x_0, x_1, ..., x_{N-1}, u_0, u_1, ..., u_{N-1}
    """
    x_size = N * n
    X = z[:x_size].reshape(N, n)
    U = z[x_size:].reshape(N, m)
    return X, U

def hermite_simpson_defect(xk, uk, xk1, uk1, h):
    """
    Hermite-Simpson collocation defect:
    
$$x_{k+1} - x_k - h/6 * (f_k + 4 f_c + f_{k+1}) = 0$$

    with midpoint state/control:
    
$$x_c = 0.5(x_k + x_{k+1}) + h/8 (f_k - f_{k+1})$$

    
$$u_c = 0.5(u_k + u_{k+1})$$

    """
    fk = dynamics(xk, uk[0])
    fk1 = dynamics(xk1, uk1[0])

    xc = 0.5 * (xk + xk1) + (h / 8.0) * (fk - fk1)
    uc = 0.5 * (uk + uk1)
    fc = dynamics(xc, uc[0])

    defect = xk1 - xk - (h / 6.0) * (fk + 4.0 * fc + fk1)
    return defect

def objective(z, N, n, m, h):
    """
    Minimize integral of  $0.5 * u^2 dt$  using trapezoidal sum.
    """

```

Here is a Hermite-Simpson direct collocation version adapted to a simple cart-pole.

State:

$x = [p, \dot{p}, \theta, \dot{\theta}]$

Control:

$u = F$

Dynamics use the standard cart-pole equations for a pole of length l , cart mass m_c , pole mass m_p , and gravity g .

```
import numpy as np
from scipy.optimize import minimize
import matplotlib.pyplot as plt

def cartpole_dynamics(x, u, params):
    """
    Continuous-time cart-pole dynamics.

    State:
        x[0] = cart position p
        x[1] = cart velocity p_dot
        x[2] = pole angle theta
        x[3] = pole angular velocity theta_dot

    Control:
        u = horizontal force on cart

    Convention:
        theta = 0 means upright.
    """
    mc = params["mc"]    # cart mass
    mp = params["mp"]    # pole mass
    l = params["l"]      # pole COM length
    g = params["g"]

    p, p_dot, th, th_dot = x
    s = np.sin(th)
    c = np.cos(th)
```

```

    total_mass = mc + mp
    denom = total_mass - mp * c * c

    # Cart acceleration
    p_ddot = (u + mp * s * (l * th_dot**2 + g * c)) /
denom

    # Pole angular acceleration
    th_ddot = (-u * c - mp * l * th_dot**2 * c * s -
total_mass * g * s) / (l * denom)

    return np.array([p_dot, p_ddot, th_dot, th_ddot])

def unpack_decision_vars(z, N, n, m):
    x_size = N * n
    X = z[:x_size].reshape(N, n)
    U = z[x_size:].reshape(N, m)
    return X, U

def hermite_simpson_defect(xk, uk, xk1, uk1, h,
params):
    fk = cartpole_dynamics(xk, uk[0], params)
    fk1 = cartpole_dynamics(xk1, uk1[0], params)

    xc = 0.5 * (xk + xk1) + (h / 8.0) * (fk - fk1)
    uc = 0.5 * (uk + uk1)
    fc = cartpole_dynamics(xc, uc[0], params)

    return xk1 - xk - (h / 6.0) * (fk + 4.0 * fc + fk1)

def angle_wrap(a):
    return (a + np.pi) % (2 * np.pi) - np.pi

def objective(z, N, n, m, h, weights, x_goal):
    """

```

```

Running cost:
    0.5 * [ w_p (p-p*)^2 + w_v p_dot^2 + w_th *
angle_error^2
           + w_w th_dot^2 + w_u u^2 ]
Terminal cost:
    0.5 * [ wf_p (p-p*)^2 + wf_v p_dot^2 + wf_th *
angle_error^2
           + wf_w th_dot^2 ]
"""
X, U = unpack_decision_vars(z, N, n, m)

w_p = weights["w_p"]
w_v = weights["w_v"]
w_th = weights["w_th"]
w_w = weights["w_w"]
w_u = weights["w_u"]

wf_p = weights["wf_p"]
wf_v = weights["wf_v"]
wf_th = weights["wf_th"]
wf_w = weights["wf_w"]

cost = 0.0

for k in range(N - 1):
    p, p_dot, th, th_dot = X[k]
    u = U[k, 0]

    th_err = angle_wrap(th - x_goal[2])
    p_err = p - x_goal[0]

    running = 0.5 * (
        w_p * p_err**2
        + w_v * p_dot**2
        + w_th * th_err**2
        + w_w * th_dot**2
        + w_u * u**2
    )
    cost += h * running

```



```

# terminal cost
p, p_dot, th, th_dot = X[-1]
th_err = angle_wrap(th - x_goal[2])
p_err = p - x_goal[0]
terminal = 0.5 * (
    wf_p * p_err**2
    + wf_v * p_dot**2
    + wf_th * th_err**2
    + wf_w * th_dot**2
)
cost += terminal

return cost

def constraint_function(z, N, n, m, h, x0, params):
    """
    Equality constraints:
    1) initial state fixed
    2) Hermite-Simpson dynamics defects
    """
    X, U = unpack_decision_vars(z, N, n, m)

    cons = []

    # Initial condition
    cons.extend(X[0] - x0)

    # Collocation defects
    for k in range(N - 1):
        defect = hermite_simpson_defect(X[k], U[k], X[k
+ 1], U[k + 1], h, params)
        cons.extend(defect)

    return np.array(cons)

def solve_cartpole_collocation():
    # dimensions
    n = 4

```

```

m = 1
N = 61
T = 2.0
h = T / (N - 1)

# cart-pole parameters
params = {
    "mc": 1.0,
    "mp": 0.1,
    "l": 0.5,
    "g": 9.81,
}

# initial state: near hanging down
x0 = np.array([0.0, 0.0, np.pi, 0.0])

# target: upright at origin
x_goal = np.array([0.0, 0.0, 0.0, 0.0])

# cost weights
weights = {
    "w_p": 1.0,
    "w_v": 0.1,
    "w_th": 20.0,
    "w_w": 1.0,
    "w_u": 0.01,
    "wf_p": 20.0,
    "wf_v": 5.0,
    "wf_th": 200.0,
    "wf_w": 20.0,
}

# initial guess
X_guess = np.zeros((N, n))
for k in range(N):
    alpha = k / (N - 1)
    X_guess[k, 0] = 0.0
    X_guess[k, 1] = 0.0
    # linearly interpolate angle from pi to 0 as a
    crude guess

```

```

        X_guess[k, 2] = (1 - alpha) * np.pi
        X_guess[k, 3] = 0.0

    U_guess = np.zeros((N, m))

    z0 = np.concatenate([X_guess.ravel(),
        U_guess.ravel()])

    # bounds
    bounds = []

    # state bounds
    p_max = 3.0
    v_max = 10.0
    w_max = 20.0
    for _ in range(N):
        bounds.append((-p_max, p_max))           # p
        bounds.append((-v_max, v_max))           # p_dot
        bounds.append((None, None))              # theta
        bounds.append((-w_max, w_max))           # theta_dot

    # control bounds
    u_max = 30.0
    for _ in range(N):
        bounds.append((-u_max, u_max))

    constraints = {
        "type": "eq",
        "fun": lambda z: constraint_function(z, N, n,
m, h, x0, params),
    }

    result = minimize(
        fun=lambda z: objective(z, N, n, m, h, weights,
x_goal),
        x0=z0,
        method="SLSQP",
        bounds=bounds,
        constraints=constraints,

```

```

        options={"ftol": 1e-7, "maxiter": 1000, "disp":
True},
    )

    X_opt, U_opt = unpack_decision_vars(result.x, N, n,
m)

    t = np.linspace(0.0, T, N)

    print("\nSuccess:", result.success)
    print("Message:", result.message)
    print("Optimal cost:", result.fun)
    print("Final state:", X_opt[-1])

    # plot
    fig, axs = plt.subplots(5, 1, figsize=(8, 12),
sharex=True)

    axs[0].plot(t, X_opt[:, 0])
    axs[0].set_ylabel("cart pos")

    axs[1].plot(t, X_opt[:, 1])
    axs[1].set_ylabel("cart vel")

    axs[2].plot(t, X_opt[:, 2])
    axs[2].set_ylabel("theta")

    axs[3].plot(t, X_opt[:, 3])
    axs[3].set_ylabel("theta_dot")

    axs[4].plot(t, U_opt[:, 0])
    axs[4].set_ylabel("force")
    axs[4].set_xlabel("time [s]")

    plt.tight_layout()
    plt.show()

    return t, X_opt, U_opt, result

if __name__ == "__main__":

```

```
solve_cartpole_collocation()
```

There is a shooting method which is a second order method: linear quadratic regulator which is a quadratic cost and linear dynamics.

LQR:

Constrained Optimization problem:

Unconstrained Optimization problem after substitution:

$$\min_{u_1, u_2, \dots, u_T} c(x_1, u_1) = c(f(x_1, u_1), u_2) + \dots + c(f(f(\dots)\dots), u_T)$$

Linear Dynamics Case:

$$f(x_t, u_t) = F_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + f_t$$

subject to the quadratic constraint:

$$c(x_t, u_t) = \frac{1}{2} \begin{bmatrix} x_t \\ u_t \end{bmatrix}^T C_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + \begin{bmatrix} x_t \\ u_t \end{bmatrix}^T c_t$$

We are regulating the trajectory with linear dynamics equation:

Base case, solve for the last u_t which is u_T by setting $t=T$ and solve as a function of the last term.

$$\begin{array}{c}
 \mathbf{x}_T \text{ (unknown)} \\
 \hline
 + \underbrace{c(f(f(\dots)\dots), \mathbf{u}_T)} \\
 \hline
 \text{only term that} \\
 \text{depends on } \mathbf{u}_T
 \end{array}$$

The last state x_t is unknown. We can express the optimization fn in terms of the last state x_T only.

The only portion of function $c(x,u)$ is u_t at the last time step:

$$Q(x_t, u_t) = const + \frac{1}{2} \begin{bmatrix} x_t \\ u_t \end{bmatrix}^T C_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + \begin{bmatrix} x_t \\ u_t \end{bmatrix}^T c_t$$

for all t .

convert the above to the terminal state quadratic by replacing t with final state T:

$$Q(x_T, u_T) = \text{const} + \frac{1}{2} \begin{bmatrix} x_T \\ u_T \end{bmatrix}^T C_t \begin{bmatrix} x_T \\ u_T \end{bmatrix} + \begin{bmatrix} x_T \\ u_T \end{bmatrix}^T c_T \quad \text{for } t=T. \text{ Look at 1 time step.}$$

break apart C_t and c_t into parts which influence u_t and parts which influence x_t :

$$C_T = \begin{bmatrix} C_{x_T, x_T} & C_{x_T, u_T} \\ C_{u_T, x_T} & C_{u_T, u_T} \end{bmatrix}$$

$$c_T = \begin{bmatrix} c_{x_T} \\ c_{u_T} \end{bmatrix}$$

$$Q(x_T, u_T) = \text{const} + \frac{1}{2} \begin{bmatrix} x_T \\ u_T \end{bmatrix}^T \begin{bmatrix} C_{x_T, x_T} & C_{x_T, u_T} \\ C_{u_T, x_T} & C_{u_T, u_T} \end{bmatrix} \begin{bmatrix} x_T \\ u_T \end{bmatrix} + \begin{bmatrix} x_T \\ u_T \end{bmatrix}^T \begin{bmatrix} c_{x_T} \\ c_{u_T} \end{bmatrix}.$$

$$\begin{bmatrix} C_{x_T, x_T} & C_{x_T, u_T} \\ C_{u_T, x_T} & C_{u_T, u_T} \end{bmatrix} \begin{bmatrix} x_T \\ u_T \end{bmatrix} = \begin{bmatrix} C_{x_T, x_T} x_T + C_{x_T, u_T} u_T \\ C_{u_T, x_T} x_T + C_{u_T, u_T} u_T \end{bmatrix}$$

Now left-multiply by $\begin{bmatrix} x_T \\ u_T \end{bmatrix}^T$:

$$\begin{bmatrix} x_T \\ u_T \end{bmatrix}^T C_T \begin{bmatrix} x_T \\ u_T \end{bmatrix} = x_T^T C_{x_T, x_T} x_T + x_T^T C_{x_T, u_T} u_T + u_T^T C_{u_T, x_T} x_T + u_T^T C_{u_T, u_T} u_T.$$

Linear term becomes:

$$\begin{bmatrix} x_T \\ u_T \end{bmatrix}^T c_T = x_T^T c_{x_T} + u_T^T c_{u_T}.$$

$$Q(x_T, u_T) = \text{const} + \frac{1}{2} x_T^T C_{x_T, x_T} x_T + \frac{1}{2} x_T^T C_{x_T, u_T} u_T + \frac{1}{2} u_T^T C_{u_T, x_T} x_T + \frac{1}{2} u_T^T C_{u_T, u_T} u_T + x_T^T c_{x_T} + u_T^T c_{u_T}.$$

Separate terms involving u_T

Since x_T is fixed, the terms

$$\text{const} + \frac{1}{2} x_T^T C_{x_T, x_T} x_T + x_T^T c_{x_T}$$

do not affect the minimization over u_T .

So focus on the u_T -dependent part:

$$Q(x_T, u_T) = \text{const} + \frac{1}{2} x_T^T C_{x_T, u_T} u_T + \frac{1}{2} u_T^T C_{u_T, x_T} x_T + \frac{1}{2} u_T^T C_{u_T, u_T} u_T + u_T^T c_{u_T}.$$

If C_T is symmetric, then

$$C_{u_T, x_T} = C_{x_T, u_T}^T,$$

and the two mixed terms combine into

$$\frac{1}{2}x_T^T C_{x_T, u_T} u_T + \frac{1}{2}u_T^T C_{u_T, x_T} x_T = u_T^T C_{u_T, x_T} x_T.$$

$$Q(x_T, u_T) = \text{const} + u_T^T C_{u_T, x_T} x_T + \frac{1}{2}u_T^T C_{u_T, u_T} u_T + u_T^T c_{u_T}.$$

Differentiate with respect to u_T

$$\text{Take the gradient: } \nabla_{u_T} Q(x_T, u_T) = C_{u_T, x_T} x_T + C_{u_T, u_T} u_T + c_{u_T}.$$

Set it equal to zero:

$$C_{u_T, x_T} x_T + C_{u_T, u_T} u_T + c_{u_T} = 0.$$

Solve for u_T :

$$C_{u_T, u_T} u_T = - (C_{u_T, x_T} x_T + c_{u_T}).$$

$$u_T = - C_{u_T, u_T}^{-1} (C_{u_T, x_T} x_T + c_{u_T}).$$

This is called the terminal optimal control law:

$$u_T = K_T x_T + k_T$$

$$K_T = - C_{u_T, u_T}^{-1} C_{u_T, x_T}, \quad k_T = - C_{u_T, u_T}^{-1} c_{u_T}.$$

This is not a fixed value, depends on x.

Plugin the derivation of u_t in terms of x_t and we get:

$$V(x_T) = \text{const} + \frac{1}{2} \begin{bmatrix} x_T \\ K_T x_T + k_T \end{bmatrix}^T C_t \begin{bmatrix} x_T \\ K_T x_T + k_T \end{bmatrix} + \begin{bmatrix} x_T \\ K_T x_T + k_T \end{bmatrix}^T c_T$$

expand the middle product:

$$\begin{bmatrix} C_{xx} & C_{xu} \\ C_{ux} & C_{uu} \end{bmatrix} \begin{bmatrix} x \\ Kx + k \end{bmatrix} = \begin{bmatrix} C_{xx}x + C_{xu}(Kx + k) \\ C_{ux}x + C_{uu}(Kx + k) \end{bmatrix}.$$

Now left-multiply by $\begin{bmatrix} x \\ Kx + k \end{bmatrix}^T$:

$$\begin{bmatrix} x \\ Kx + k \end{bmatrix}^T \begin{bmatrix} C_{xx} & C_{xu} \\ C_{ux} & C_{uu} \end{bmatrix} \begin{bmatrix} x \\ Kx + k \end{bmatrix} \\ = x^T C_{xx} x + x^T C_{xu} (Kx + k) + (Kx + k)^T C_{ux} x + (Kx + k)^T C_{uu} (Kx + k).$$

$$V(x) = \text{const} + \frac{1}{2} x^T C_{xx} x + \frac{1}{2} x^T C_{xu} (Kx + k) + \frac{1}{2} (Kx + k)^T C_{ux} x \\ + \frac{1}{2} (Kx + k)^T C_{uu} (Kx + k) + x^T c_x + (Kx + k)^T c_u.$$

Expand each term separately

Term A

$$\frac{1}{2}x^T C_{xx}x$$

Term B

$$\frac{1}{2}x^T C_{xu}(Kx + k) = \frac{1}{2}x^T C_{xu}Kx + \frac{1}{2}x^T C_{xu}k$$

Term C

$$\frac{1}{2}(Kx + k)^T C_{ux}x = \frac{1}{2}x^T K^T C_{ux}x + \frac{1}{2}k^T C_{ux}x$$

Term D

$$\frac{1}{2}(Kx + k)^T C_{uu}(Kx + k)$$

Expanding:

$$(Kx + k)^T C_{uu}(Kx + k) = x^T K^T C_{uu}Kx + x^T K^T C_{uu}k + k^T C_{uu}Kx + k^T C_{uu}k$$

$$\frac{1}{2}(Kx + k)^T C_{uu}(Kx + k) = \frac{1}{2}x^T K^T C_{uu}Kx + \frac{1}{2}x^T K^T C_{uu}k + \frac{1}{2}k^T C_{uu}Kx + \frac{1}{2}k^T C_{uu}k$$

Term E

$$x^T c_x$$

Term F

$$(Kx + k)^T c_u = x^T K^T c_u + k^T c_u$$

$$\begin{aligned} V(x) = \text{const} &+ \frac{1}{2} x^T C_{xx} x + \frac{1}{2} x^T C_{xu} K x + \frac{1}{2} x^T C_{xu} k \\ &+ \frac{1}{2} x^T K^T C_{ux} x + \frac{1}{2} k^T C_{ux} x \\ &+ \frac{1}{2} x^T K^T C_{uu} K x + \frac{1}{2} x^T K^T C_{uu} k + \frac{1}{2} k^T C_{uu} K x + \frac{1}{2} k^T C_{uu} k \\ &+ x^T c_x + x^T K^T c_u + k^T c_u. \end{aligned}$$

Collect quadratic-in-x, linear-in-x, and constant terms
Quadratic terms

$$\frac{1}{2} x^T C_{xx} x + \frac{1}{2} x^T C_{xu} K x + \frac{1}{2} x^T K^T C_{ux} x + \frac{1}{2} x^T K^T C_{uu} K x$$

Factor out $\frac{1}{2} x^T (\dots) x$:

$$\frac{1}{2}x^T (C_{xx} + C_{xu}K + K^T C_{ux} + K^T C_{uu}K) x$$

Linear Terms:

$$\frac{1}{2}x^T C_{xu}k + \frac{1}{2}k^T C_{ux}x + \frac{1}{2}x^T K^T C_{uu}k + \frac{1}{2}k^T C_{uu}Kx + x^T c_x + x^T K^T c_u$$

Rewrite the transpose forms

$$k^T C_{ux}x = x^T C_{ux}^T k, \quad k^T C_{uu}Kx = x^T K^T C_{uu}^T k$$

If C_T is symmetric, then $C_{ux}=C_{xu}^T$ and $C_{uu}=C_{uu}^T$. So these pairs combine:

$$\frac{1}{2}x^T C_{xu}k + \frac{1}{2}k^T C_{ux}x = x^T C_{xu}k$$

$$\frac{1}{2}x^T K^T C_{uu}k + \frac{1}{2}k^T C_{uu}Kx = x^T K^T C_{uu}k$$

linear term is

$$x^T (c_x + C_{xu}k + K^T c_u + K^T C_{uu}k)$$

Constant terms

$$\text{const} + \frac{1}{2}k^T C_{uu}k + k^T c_u$$

$$V(x) = \text{const} + \frac{1}{2}x^T Vx + x^T v$$

$$V = C_{xx} + C_{xu}K + K^T C_{ux} + K^T C_{uu}K$$

$$v = c_x + C_{xu}k + K^T c_u + K^T C_{uu}k.$$

$$V(x_T) = \text{const} + \frac{1}{2}x_T^T V_T x_T + x_T^T v_T$$

where"

$$V_T = C_{x_T, x_T} + C_{x_T, u_T} K_T + K_T^T C_{u_T, x_T} + K_T^T C_{u_T, u_T} K_T$$

and:

$$v_T = c_{x_T} + C_{x_T, u_T} k_T + K_T^T c_{u_T} + K_T^T C_{u_T, u_T} k_T.$$

Now lets think of the previous time steps:
how to propagate this solution for t=T to t=0?

Start at time T-1

solve for u_{T-1} in terms of x_{T-1}

u_{T-1} is governed by dynamics equations

$$f(x_{T-1}, u_{T-1}) = x_T = F_{T-1} \begin{bmatrix} x_{T-1} \\ u_{T-1} \end{bmatrix}^T + f_{T-1}$$

$$Q(x_{T-1}, u_{T-1}) = \text{const} + \frac{1}{2} C_{T-1} \begin{bmatrix} x_{T-1} \\ u_{T-1} \end{bmatrix} + \begin{bmatrix} x_{T-1} \\ u_{T-1} \end{bmatrix}^T c_{T-1} + V(f(x_{T-1}, u_{T-1}))$$

Let

$$z_{T-1} := \begin{bmatrix} x_{T-1} \\ u_{T-1} \end{bmatrix},$$

$$x_T = F_{T-1}z_{T-1} + f_{T-1}.$$

From dynamics equations:

$$V(f(x_{T-1}, u_{T-1})) = V(x_T) = \text{const} + \frac{1}{2}x_T^T V_T x_T + X_T^T v_T$$

$$x_T = F_{T-1}z_{T-1} + f_{T-1}.$$

$$V(x_T) = \text{const} + \frac{1}{2}(F_{T-1}z_{T-1} + f_{T-1})^T V_T (F_{T-1}z_{T-1} + f_{T-1}) + (F_{T-1}z_{T-1} + f_{T-1})^T v_T.$$

Expand Quadratic Term:

$$\frac{1}{2}(Fz + f)^T V_T (Fz + f)$$

with: $z = z_{T-1}, F = F_{T-1}, f = f_{T-1}$:

$$\frac{1}{2}(Fz + f)^T V_T (Fz + f) = \frac{1}{2}z^T F^T V_T Fz + \frac{1}{2}z^T F^T V_T f + \frac{1}{2}f^T V_T Fz + \frac{1}{2}f^T V_T f.$$

2 terms after the equal sign:

$$z^T F^T V_T f = f^T V_T Fz$$

Proof:

taking transpose

$$(z^T F^T V_T f)^T = f^T V_T^T F z.$$

If V_T is symmetric, $V_T^T = V_T$, so

$$(z^T F^T V_T f)^T = f^T V_T F z.$$

Hence

$$z^T F^T V_T f = f^T V_T F z.$$

$$\frac{1}{2} z^T F^T V_T f + \frac{1}{2} f^T V_T F z = z^T F^T V_T f.$$

$$\frac{1}{2} (Fz + f)^T V_T (Fz + f) = \frac{1}{2} z^T F^T V_T F z + z^T F^T V_T f + \frac{1}{2} f^T V_T f.$$

$$= \frac{1}{2} (Fz + f)^T V_T (Fz + f) = \frac{1}{2} z^T F^T V_T F z + z^T F^T V_T f + \text{const.}$$

Expand the linear term

$$(Fz + f)^T v_T = z^T F^T v_T + f^T v_T.$$

The second term is constant, so

$$(Fz + f)^T v_T = z^T F^T v_T + \text{const.}$$

Put the value function together

$$V(x_T) = \text{const} + \frac{1}{2} z_{T-1}^\top F_{T-1}^\top V_T F_{T-1} z_{T-1} + z_{T-1}^\top F_{T-1}^\top V_T f_{T-1} + z_{T-1}^\top F_{T-1}^\top v_T.$$

Substitute back into Q

$$Q(x_{T-1}, u_{T-1}) = \text{const} + \frac{1}{2} z_{T-1}^\top C_{T-1} z_{T-1} + z_{T-1}^\top c_{T-1} + \frac{1}{2} z_{T-1}^\top F_{T-1}^\top V_T F_{T-1} z_{T-1} + z_{T-1}^\top F_{T-1}^\top V_T f_{T-1} + z_{T-1}^\top F_{T-1}^\top v_T.$$

Group quadratic and linear terms:

$$Q(x_{T-1}, u_{T-1}) = \text{const} + \frac{1}{2} z_{T-1}^\top Q_{T-1} z_{T-1} + z_{T-1}^\top q_{T-1},$$

where

$$Q_{T-1} = C_{T-1} + F_{T-1}^\top V_T F_{T-1},$$

and

$$q_{T-1} = c_{T-1} + F_{T-1}^\top V_T f_{T-1} + F_{T-1}^\top v_T.$$

$$Q(x_{T-1}, u_{T-1}) = \text{const} + \frac{1}{2} \begin{bmatrix} x_{T-1} \\ u_{T-1} \end{bmatrix}^\top Q_{T-1} \begin{bmatrix} x_{T-1} \\ u_{T-1} \end{bmatrix} + \begin{bmatrix} x_{T-1} \\ u_{T-1} \end{bmatrix}^\top q_{T-1},$$

Now differentiate with respect to u_{T-1} :

$$\nabla_{u_{T-1}} Q(x_{T-1}, u_{T-1}) = Q_{ux}x_{T-1} + Q_{uu}u_{T-1} + q_u.$$

Set it equal to zero:

$$Q_{ux}x_{T-1} + Q_{uu}u_{T-1} + q_u = 0.$$

Solve for u_{T-1} :

$$Q_{uu}u_{T-1} = -(Q_{ux}x_{T-1} + q_u),$$

$$u_{T-1} = -Q_{uu}^{-1}(Q_{ux}x_{T-1} + q_u).$$

define:

$$K_{T-1} = -Q_{uu}^{-1}Q_{ux}, \quad k_{T-1} = -Q_{uu}^{-1}q_u.$$

Then the optimal control law is

$$u_{T-1} = K_{T-1}x_{T-1} + k_{T-1}.$$

Restoring the full indexed notation:

$$K_{T-1} = -Q_{u_{T-1}, u_{T-1}}^{-1} Q_{u_{T-1}, x_{T-1}}, \quad k_{T-1} = -Q_{u_{T-1}, u_{T-1}}^{-1} q_{u_{T-1}},$$

$$u_{T-1} = K_{T-1} x_{T-1} + k_{T-1}.$$

That is the affine feedback law at step T-1.

Time iterate back to t=0.

Some background: LQR is a MDP. The state and action are continuous. The dynamics are transition functions and the cost is quadratic. The next state depends on current state and action.

A finite horizon Riccati backward recursion is linear/quadratic.

The updates for Q_t and q_t match the control law:

$$u_t = K_t x_t + k_t.$$

and the value function updates

$$V(x_t) = \text{const} + \frac{1}{2} x_t^T V_t x_t + X_t^T v_t$$

t=T to 1:

$$Q_t = C_t + F_t^T V_{t+1} F_t$$

$$q_t = c_t + F_t^T V_{t+1} f_t + F_t^T v_{t+1}$$

$$Q(x_t, u_t) = \text{const} + \frac{1}{2} \begin{bmatrix} x_t \\ u_t \end{bmatrix}^T Q_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + \begin{bmatrix} x_t \\ u_t \end{bmatrix}^T q_t,$$

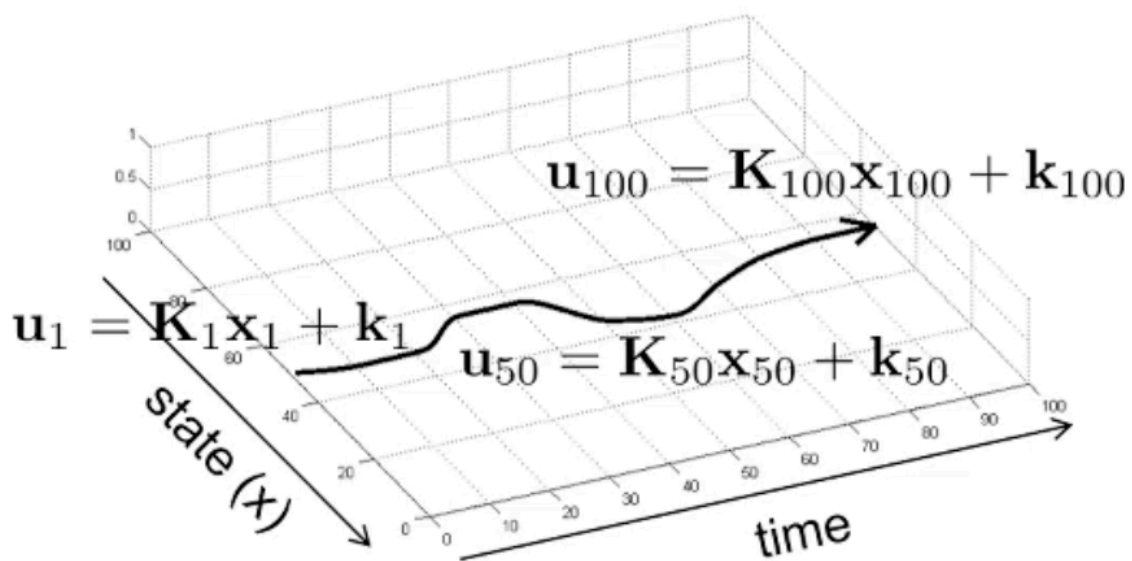
$$u_t < - \operatorname{argmin} Q(x_t, u_t) = K_t x_t + k_t$$

$$K_t = -Q_{uu}^{-1}Q_{ux}, \quad k_t = -Q_{uu}^{-1}q_u.$$

$$V_t = Q_{x_t, x_t} + Q_{x_t, u_t}K_t + K_t^T Q_{u_t, x_t} + K_t^T Q_{u_t, u_t}K_t$$

$$v_t = q_{x_t} + Q_{x_t, u_t}k_t + K_t^T Q_{u_t} + K_t^T Q_{u_t, u_t}k_t.$$

$$V(x_t) = \text{const} + \frac{1}{2}x_t^T V_t x_t + x_t^T v_t$$



⌈ T

Once we know beginning $t=1$ we can do forward pass and calculate $t=2$, etc...

for t=1 to T:

$$u_t = K_t x_t + k_t.$$

$$x_{t+1} = f(x_t, u_t)$$

LQR Code:

Show examples of the forward and backward passes.

Nonlinear case: DDP/iterative LQR

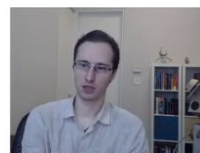
Linear-quadratic assumptions:

$$f(\mathbf{x}_t, \mathbf{u}_t) = \mathbf{F}_t \begin{bmatrix} \mathbf{x}_t \\ \mathbf{u}_t \end{bmatrix} + \mathbf{f}_t \quad c(\mathbf{x}_t, \mathbf{u}_t) = \frac{1}{2} \begin{bmatrix} \mathbf{x}_t \\ \mathbf{u}_t \end{bmatrix}^T \mathbf{C}_t \begin{bmatrix} \mathbf{x}_t \\ \mathbf{u}_t \end{bmatrix} + \begin{bmatrix} \mathbf{x}_t \\ \mathbf{u}_t \end{bmatrix}^T \mathbf{c}_t$$

Can we *approximate* a nonlinear system as a linear-quadratic system?

$$f(\mathbf{x}_t, \mathbf{u}_t) \approx f(\hat{\mathbf{x}}_t, \hat{\mathbf{u}}_t) + \nabla_{\mathbf{x}_t, \mathbf{u}_t} f(\hat{\mathbf{x}}_t, \hat{\mathbf{u}}_t) \begin{bmatrix} \mathbf{x}_t - \hat{\mathbf{x}}_t \\ \mathbf{u}_t - \hat{\mathbf{u}}_t \end{bmatrix}$$

$$c(\mathbf{x}_t, \mathbf{u}_t) \approx c(\hat{\mathbf{x}}_t, \hat{\mathbf{u}}_t) + \nabla_{\mathbf{x}_t, \mathbf{u}_t} c(\hat{\mathbf{x}}_t, \hat{\mathbf{u}}_t) \begin{bmatrix} \mathbf{x}_t - \hat{\mathbf{x}}_t \\ \mathbf{u}_t - \hat{\mathbf{u}}_t \end{bmatrix} + \frac{1}{2} \begin{bmatrix} \mathbf{x}_t - \hat{\mathbf{x}}_t \\ \mathbf{u}_t - \hat{\mathbf{u}}_t \end{bmatrix}^T \nabla_{\mathbf{x}_t, \mathbf{u}_t}^2 c(\hat{\mathbf{x}}_t, \hat{\mathbf{u}}_t) \begin{bmatrix} \mathbf{x}_t - \hat{\mathbf{x}}_t \\ \mathbf{u}_t - \hat{\mathbf{u}}_t \end{bmatrix}$$



Lets do this in 2 steps, first consider linear and stochastic then nonlinear and stochastic.

Add stochastic:

$$f(x_t, u_t) = F_t \begin{bmatrix} x_t \\ u_t \end{bmatrix}^T + f_t; \text{ linear dynamics}$$

$x_{t+1} \sim p(x_{t+1} | x_t, u_t)$ stochastic, in special case of gaussian, we have mean and variance given below:

$$p(x_{t+1} | x_t, u_t) = N(F_t + \begin{bmatrix} x_t \\ u_t \end{bmatrix}^T + f_t, \sum_t)$$

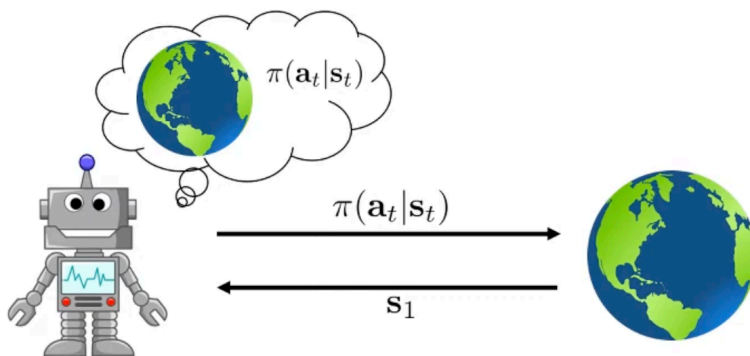
Choose actions $u_t = K_t x_t + k_t$ This looks like the same answer as before. He didnt derive this.

Gaussian is symmetric, adding gaussian wont change solution. But it does change the states. the states are stochastic, the policy $K_t x_t + k_t$ is a controller or policy. This is a closed loop policy in the linear quadratic gaussian case.

no change to algorithm.

can ignore $\sum_t$ from gaussian symmetry. Rederive on own.

The stochastic closed-loop case



form of π ?

$$p(s_1, a_1, \dots, s_T, a_T) = p(s_1) \prod_{t=1}^T \pi(a_t | s_t) p(s_{t+1} | s_t, a_t)$$

time-varying linear

$$K_t s_t + k_t$$

$$\pi = \arg \max_{\pi} E_{\tau \sim p(\tau)} \left[\sum_t r(s_t, a_t) \right]$$

The policy is a linear time varying policy. Not a LLM, not a MLP. But a different K_t and k_t in each timestep.

K_t is the feedback gain
 k_t is the forward update
 s_t are the states

We assume we know the dynamics model. If we don't know the dynamics model we can replace with a fn approximator in a NN. This is not a policy. Then we linearize and take the derivatives using backprop.

1. Collect transitions

From the real system or simulator, gather data

(x_t, u_t, x_{t+1})

2. Train neural dynamics model

Fit

$\hat{x}_{t+1} = f_{\theta}(x_t, u_t)$

by supervised learning.

3. Linearize the neural model along a nominal trajectory

At the current rollout,

$A_t = \frac{\partial f_{\theta}}{\partial x_t}, \quad \quad$
 $B_t = \frac{\partial f_{\theta}}{\partial u_t}.$

With a neural net, these can be computed by autodiff.

4. Quadratize cost

Compute local cost derivatives as usual.

5. Run iLQG backward/forward pass

Using the learned local linearization and cost approximation, compute:

- feedforward update k_t
- feedback gain K_t

6. Execute controls

Apply the optimized trajectory/controller to the real system or simulator.

7. Gather more data and repeat

This improves the model and the controller over time.

What changes mathematically

Nothing fundamental in the iLQG equations changes.

You still use local approximations:

$$\begin{aligned} x_{t+1} &\approx \hat{f}_t(\bar{x}_t, \bar{u}_t) \\ &+ A_t(x_t - \bar{x}_t) \\ &+ B_t(u_t - \bar{u}_t). \end{aligned}$$

Back to the YT lecture:

Add nonlinearity. Replace nonlinearity with Taylor series expansion.

Start with what we know, the linear quadratic form is linear dynamics with quadratic cost:

Linear Dynamics:

$$f(x_t, u_t) = F_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + f_t$$

Quadratic cost:

$$c(x_t, u_t) = \frac{1}{2} \begin{bmatrix} x_t \\ u_t \end{bmatrix}^T C_t \begin{bmatrix} x_t \\ u_t \end{bmatrix} + \begin{bmatrix} x_t \\ u_t \end{bmatrix}^T c_t$$

TAYLOR SERIES:

$$f(x_t, u_t) \sim f(\hat{x}_t, \hat{u}_t) + \nabla_{x_t, u_t} f(\hat{x}_t, \hat{u}_t) \begin{bmatrix} x_t - \hat{x}_t \\ u_t - \hat{u}_t \end{bmatrix}$$

$$c(x_t, u_t) \sim c(\hat{x}_t, \hat{u}_t) + \nabla c(\hat{x}_t, \hat{u}_t) \begin{bmatrix} x_t - \hat{x}_t \\ u_t - \hat{u}_t \end{bmatrix} + \frac{1}{2} \begin{bmatrix} x_t - \hat{x}_t \\ u_t - \hat{u}_t \end{bmatrix} \nabla^2 c(\hat{x}_t, \hat{u}_t) \begin{bmatrix} x_t - \hat{x}_t \\ u_t - \hat{u}_t \end{bmatrix}^T c_t$$

$$\bar{f}(\delta x_t, \delta u_t) = F_t \begin{bmatrix} \delta x_t \\ \delta u_t \end{bmatrix}$$

$$c(\delta x_t, \delta u_t) = \frac{1}{2} \begin{bmatrix} \delta x_t \\ \delta u_t \end{bmatrix}^T C_t \begin{bmatrix} \delta x_t \\ \delta u_t \end{bmatrix} + \begin{bmatrix} \delta x_t \\ \delta u_t \end{bmatrix}^T c_t$$

$$\bar{f}(\delta \mathbf{x}_t, \delta \mathbf{u}_t) = \underbrace{\mathbf{F}_t}_{\nabla_{\mathbf{x}_t, \mathbf{u}_t} f(\hat{\mathbf{x}}_t, \hat{\mathbf{u}}_t)} \begin{bmatrix} \delta \mathbf{x}_t \\ \delta \mathbf{u}_t \end{bmatrix}$$

$$\bar{c}(\delta \mathbf{x}_t, \delta \mathbf{u}_t) = \frac{1}{2} \begin{bmatrix} \delta \mathbf{x}_t \\ \delta \mathbf{u}_t \end{bmatrix}^T \underbrace{\mathbf{C}_t}_{\nabla_{\mathbf{x}_t, \mathbf{u}_t}^2 c(\hat{\mathbf{x}}_t, \hat{\mathbf{u}}_t)} \begin{bmatrix} \delta \mathbf{x}_t \\ \delta \mathbf{u}_t \end{bmatrix} + \begin{bmatrix} \delta \mathbf{x}_t \\ \delta \mathbf{u}_t \end{bmatrix}^T \underbrace{\mathbf{c}_t}_{\nabla_{\mathbf{x}_t, \mathbf{u}_t} c(\hat{\mathbf{x}}_t, \hat{\mathbf{u}}_t)}$$

Iterative LQR:

until convergence:

$$\mathbf{F}_t = \nabla_{x_t, u_t} f(\hat{x}_t, \hat{u}_t)$$

$$\mathbf{c}_t = \nabla_{x_t, u_t} c(\hat{x}_t, \hat{u}_t)$$

$$\mathbf{C}_t = \nabla_{x_t, u_t}^2 c(\hat{x}_t, \hat{u}_t)$$

Run LQR backwards pass on δx_t and δu_t

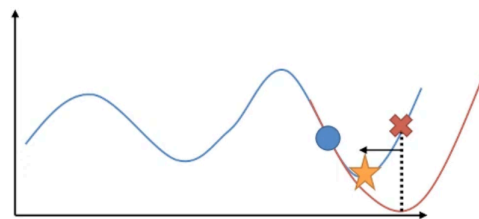
Run forward pass w nonlinear dynamics on $u_t = K_t(x - \hat{x}_t) + k_t + \hat{u}_t$
update x_t and u_t

Improvement:

Nonlinear case: DDP/iterative LQR

$$\hat{\mathbf{x}} \leftarrow \arg \min_{\mathbf{x}} \frac{1}{2} (\mathbf{x} - \hat{\mathbf{x}})^T \mathbf{H} (\mathbf{x} - \hat{\mathbf{x}}) + \mathbf{g}^T (\mathbf{x} - \hat{\mathbf{x}})$$

why is this a bad idea?



until convergence:

$$\mathbf{F}_t = \nabla_{\mathbf{x}_t, \mathbf{u}_t} f(\hat{\mathbf{x}}_t, \hat{\mathbf{u}}_t)$$

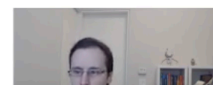
$$\mathbf{c}_t = \nabla_{\mathbf{x}_t, \mathbf{u}_t} c(\hat{\mathbf{x}}_t, \hat{\mathbf{u}}_t)$$

$$\mathbf{C}_t = \nabla_{\mathbf{x}_t, \mathbf{u}_t}^2 c(\hat{\mathbf{x}}_t, \hat{\mathbf{u}}_t)$$

Run LQR backward pass on state $\delta \mathbf{x}_t = \mathbf{x}_t - \hat{\mathbf{x}}_t$ and action $\delta \mathbf{u}_t = \mathbf{u}_t - \hat{\mathbf{u}}_t$

Run forward pass with $\mathbf{u}_t = \mathbf{K}_t(\mathbf{x}_t - \hat{\mathbf{x}}_t) + \alpha \mathbf{k}_t + \hat{\mathbf{u}}_t$

Update $\hat{\mathbf{x}}_t$ and $\hat{\mathbf{u}}_t$ based on states and actions in forward pass



add alpha

